FUNDISHOP

Sprint Retrospective and Improvement Actions

Agile team reflection | Reconstructed from available project history

Project	FundiShop – Local Services Marketplace
Student	Christian Hakizimana
Focus	Sprint activities, challenges, lessons and improvement actions
Source basis	Previous FundiShop project discussions; sprint boundaries and dates should be verified

1. Purpose of the retrospective

A sprint retrospective gives the team an opportunity to reflect on the way the work was completed, identify what helped or slowed progress, and agree on practical changes for the next sprint. This document records a retrospective for the FundiShop development journey. It focuses on team communication, planning, frontend and backend integration, database work, booking functionality, dashboards and testing.

The sprint groupings below are an organised reconstruction from the project history available in our discussions. They are not verified Trello sprint reports or formal meeting minutes. Confirm the actual sprint dates, team feedback and completed work before submitting.

Sprint 1 – Project initiation and initial interface

Period: Early project stage (exact dates to confirm)

Sprint goal: Establish the FundiShop idea, organise the work and begin the first user-facing screens.

What went well

- The team discussed the purpose of FundiShop as a platform connecting homeowners with local Fundis.
- The main user roles and core features were identified, including account access, finding a Fundi and making service bookings.
- A Trello board was used to organise project tasks and track work.
- Initial homepage and navigation development began, supported by team communication and Microsoft Teams meetings.

What did not go well / challenges

- Some meetings were cancelled or affected by poor communication and technical difficulties.
- The meeting schedule and task responsibilities were not always communicated clearly.
- Different levels of technical experience made it harder to estimate and divide some tasks.

Improvement actions

Action	How we will improve	Owner
Agree on a regular meeting schedule	Use a team poll to select a suitable meeting time and publish it in the shared communication channel.	Team
Clarify ownership of tasks	Assign an owner and expected outcome to each Trello card before work begins.	Team
Keep Trello current	Move cards as work progresses and add short notes when a task is blocked.	All members

Key lesson: A shared plan is only useful when every team member understands it and updates it regularly.

Sprint 2 – Frontend development and API integration

Period: Frontend implementation stage (exact dates to confirm)

Sprint goal: Build the main React interface and prepare it to communicate with the Spring Boot backend.

What went well

- The frontend was developed as a separate React application using Vite.
- Main navigation and user-facing pages such as the homepage, login and registration were progressed.
- The team reviewed how the frontend would call backend REST endpoints.
- The local development setup used the frontend at localhost:5173 and backend at localhost:8080.

What did not go well / challenges

- A registration request from the frontend encountered a CORS issue because the backend did not allow the frontend origin.
- Frontend and backend integration required matching request fields and response formats.
- Testing was needed to distinguish interface problems from API or server configuration problems.

Improvement actions

Action	How we will improve	Owner
Test API connections early	Verify each endpoint with a test request before connecting it to a page.	Frontend/backend developers
Document API contracts	Record endpoint paths, required fields and expected responses in a shared project note.	Team
Use a repeatable integration checklist	Check server status, CORS configuration, request payload and response before marking integration complete.	Team

Key lesson: Integrating small pieces continuously is more effective than waiting until several pages are complete.

Sprint 3 – Database, booking workflow and role-specific data

Period: Core backend and data stage (exact dates to confirm)

Sprint goal: Support the main marketplace workflow with persistent data and booking records.

What went well

- Core domain entities included User, Address, FundiProfile and Booking.
- The booking model captured the customer, Fundi profile, scheduled date and time, job description and booking status.
- The booking lifecycle included REQUESTED, ACCEPTED, DECLINED, RESCHEDULED, COMPLETED and CANCELLED.
- The team considered how bookings should be retrieved for the homeowner and Fundi experiences.

What did not go well / challenges

- Entity relationships and identifiers needed careful checking so that bookings linked to the correct user and Fundi profile.
- Incorrect or incomplete test data could make API results difficult to verify.
- The same booking needed to be represented consistently in backend logic and frontend screens.

Improvement actions

Action	How we will improve	Owner
Review the ERD before changing entities	Check primary keys, foreign keys and relationship cardinalities against the application requirements.	Database/backend developers
Prepare consistent test data	Use known homeowner, Fundi and booking records for repeatable tests.	Team
Test each booking status	Verify that each allowed status is saved and displayed consistently.	Team

Key lesson: A clear data model and repeatable test data make feature development and debugging more reliable.

Sprint 4 – Dashboards, interface refinement and testing

Period: Dashboard and refinement stage (exact dates to confirm)

Sprint goal: Make the application more useful to each role and check that the main user journeys work together.

What went well

- The homeowner dashboard included summary areas for total, pending, accepted and completed bookings.
- The team reviewed role-specific access and retrieval of customer and Fundi profile information.
- The interface was refined to make the marketplace pages and booking information easier to use.
- Subsystem testing was identified as an important activity before collecting final project evidence.

What did not go well / challenges

- Dashboard totals could be incorrect if booking records were filtered or counted incorrectly.
- The application needed to ensure that users only saw information associated with their own account and role.
- More end-to-end testing and genuine evidence screenshots were still required.

Improvement actions

Action	How we will improve	Owner
Validate dashboard figures	Compare displayed totals against the actual booking records returned by the API.	Frontend/backend developers
Test role permissions	Run the same checks with homeowner and Fundi accounts and verify the data each can access.	Team
Use a final test checklist	Record the expected result, actual result, status and any defect for each main workflow.	Team
Capture authentic evidence	Save screenshots of the working application, Trello progress and relevant code or database views.	Each member

Key lesson: A feature is not complete merely because it appears on screen; its data, permissions and full workflow must also work.

6. Overall team improvement plan

Area	Agreed improvement
Communication	Keep a consistent meeting schedule; share cancellations early; post short progress updates when a member cannot attend.
Task management	Keep Trello cards small, assign an owner, define acceptance criteria and update the board as work changes.
Technical integration	Test frontend/backend endpoints incrementally and document request/response formats.
Database quality	Maintain an updated ERD and use consistent test records for users, Fundi profiles and bookings.
Testing	Test both positive and negative cases, each booking status, role-specific access and dashboard totals.
Evidence and documentation	Capture genuine screenshots and keep them organised by sprint and feature.

7. Individual reflection

My contribution to FundiShop included coding progress, improving the user interface—particularly the Report Page—updating the ERD/database schema and carrying out subsystem testing. Working on the project showed me that software development requires more than writing code: communication, task coordination, clear data relationships and regular testing all affect the quality of the final system. I also learned the importance of checking how frontend components interact with backend endpoints and how the same information is presented to different user roles.

8. Evidence to support the retrospective

Attach genuine evidence where available. Suggested evidence includes:

- Trello board screenshots showing backlog items and task movement.
- Actual Microsoft Teams meeting records or team communication showing planning and blockers.
- Screenshots of the React homepage, registration, booking and dashboard pages.

- Relevant Spring Boot controller/service code or API test results.
- ERD and database screenshots showing the core entities and relationships.
- Subsystem test results showing test cases, expected outcomes and actual outcomes.

Verification note: This retrospective is a structured draft based on the FundiShop project history available in previous discussions. Sprint names, boundaries, team opinions, action owners and outcomes should be checked with the actual team and Trello records. Do not present reconstructed details as confirmed meeting minutes.

END OF SPRINT RETROSPECTIVE